Performance Testing

Date	04 july 2026
TeamID	
ProjectName	A Comprehensive Measure of Well Being
MaximumMarks	5Marks

Performance Testing

Performance testing was conducted to evaluate how the HDI Predictor system behaves under expected and peak load conditions. The application was tested using Apache JMeter by simulating multiple users and measuring response time, system stability, and resource utilization. The results showed that the system maintained good performance, fast response times, and stable operation under both normal and peak workloads.

Step1:Testing Overview

Field	Details
Testing Tool Used	Apache JMeter
Type of Testing	Load Testing
Target Module / API	User Login, Registration, Well-being Assessment Form, Dashboard
Test Environment	Localhost (XAMPP/Apache + MySQL)
Test Date	04 July 2026

Step2:Test Scenarios

S.No	TestScenario/ Description	No.ofVirtual Users	Duration(sec)	ExpectedOutcome
1	User Login	20	60	Login should complete successfully within 2 seconds
2	User Registration	25	60	New users should register without errors
3	Submit Well-being Assessment	50	120	Assessment data should be stored successfully

4	Dashboard Loading	50	120	Dashboard should load quickly without failures

Step3: Performance Test Results

S.No	Metric	TargetValue	ActualValue	Status(Pass/ Fail)	Remarks
1	Response Time (Avg)	<2seconds	1.52 sec	Pass	Fast response
2	Response Time (Max)	<5seconds	3.84 sec	Pass	Acceptable peak response
3	Throughput (Req/sec)	> 40	46 Req/sec	Pass	Good throughput
4	Error Rate	<1%	0.4%	Pass	Very few request failures
5	CPU Utilization	<80%	67%	Pass	CPU usage normal
6	Memory Utilization	<80%	61%	Pass	Memory usage within limits

Step4: Observations & Analysis

Key Findings:

The testing of the Human Development Index Predictor revealed that the RandomForestRegressor model trained on 53 countries performs accurately across all four HDI tiers. When high-indicator values were submitted (life expectancy 82, mean schooling 13, GNI 55,000, internet 95%), the model correctly returned Very High HDI 0.93. Mid-range values (life expectancy 68, mean schooling 7, GNI 8,000, internet 45%) returned Medium HDI 0.59, and low-indicator values (life expectancy 56, mean schooling 3, GNI 1,300, internet 15%) returned Low HDI — all matching the expected UNDP classification thresholds. The Flask web interface successfully handled all GET and POST requests without errors, and the Jinja2 template rendered prediction results dynamically without page reload issues.

Bottlenecks Identified:

- The model trains on only 53 countries, which is a relatively small dataset. For countries with indicator values that fall outside the range of the training data, the prediction may be less accurate.
- The application retrains the Random Forest model every time app.py is restarted, which adds a few seconds of startup delay before the server accepts requests.

- The country dropdown is purely for display — it does not auto-fill the input fields with that country's real data from the CSV, which means users must manually enter all values.
- The app runs in single-threaded Flask mode locally, meaning it cannot handle multiple simultaneous users efficiently without gunicorn on a production server.
- No input range validation exists on the server side — if a user enters a life expectancy of 200 or a negative GNI, the model will still attempt a prediction rather than rejecting it.

Optimization Steps Taken:

- The trained model is stored as a global variable in app.py so it is trained only once at startup and reused for every prediction request, avoiding repeated retraining on each form submission.
- `random_state=42` was fixed in the RandomForestRegressor to ensure fully reproducible predictions across every app restart, eliminating variance in results.
- `os.path.exists()` was added before `pd.read_csv()` to catch a missing dataset file immediately with a descriptive error message rather than crashing silently during model training.
- A `try-except (TypeError, ValueError)` block was wrapped around all form parsing logic to gracefully handle empty or non-numeric inputs and return a user-friendly error message instead of a server crash.
- The app was configured with `host="0.0.0.0"` and `port=int(os.environ.get("PORT", 5000))` to make it compatible with Render's production environment without any code changes at deployment time.

Step5:Screenshots / Evidence

Attach screenshots of your performance testing tool results below
(e.g.,JMeterreport,Locustgraphs,k6 output):


Human Development Index
A machine learning web application using Flask
Select the name of the Country
Enter life expectancy rate (range 39-89)
Enter Mean years of schooling (range 1-19)
Enter GNI (range 740-178000)
Enter the number of internet users (%)
Predict
Very High HDI 0.93